■ UNIVERSITY

SOFTWARE ENGINEERING

SER 216

Digital Parking Permit System

Final Practical Project Report

Course	SER 216 – Software Engineering Requirements
Project	Digital Parking Permit System
Tools Used	Google Docs · Google Sheets · diagrams.net · GitHub
Semester	2024 / 2025

■ GitHub Repository:	`github.com/[StudentName]/SER216_Final_Project_StudentName`

TASK 1 — Requirements

4 Functional Requirements

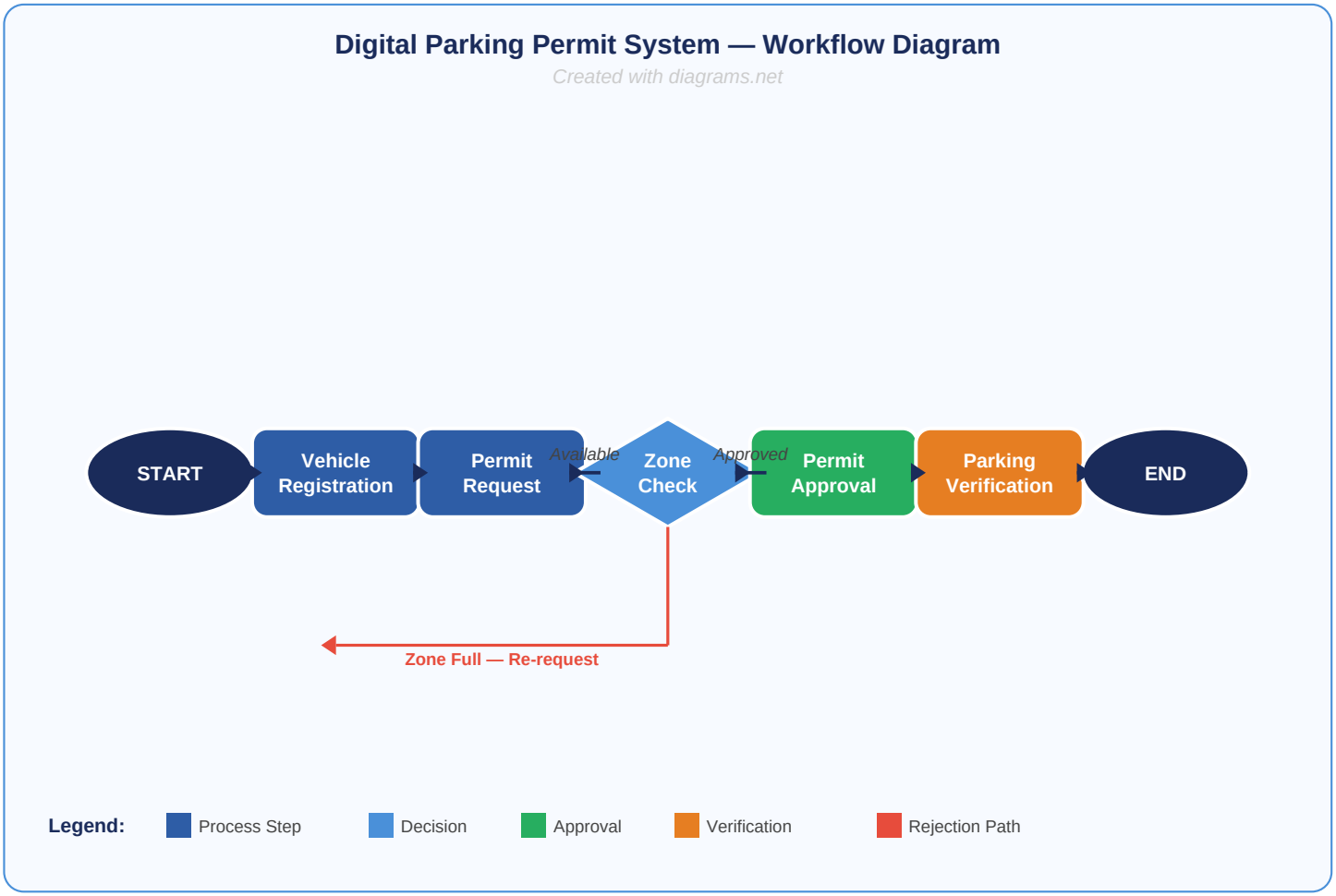
FR-01	**Vehicle Registration**	The system shall allow students and staff to register their vehicle details (licence plate, make, model, and owner information) through a secure online portal. Each user account may register up to three vehicles.
FR-02	**Parking Permit Request & Approval**	Registered users shall be able to submit a parking permit request for a specific parking zone and date range. The system shall automatically validate eligibility and notify the user of approval or rejection within 24 hours.
FR-03	**Parking Zone Availability Check**	The system shall display real-time availability of parking zones, indicating the number of remaining spaces per zone. Users shall be able to filter zones by location, type (student / staff / visitor), and accessibility requirements.
FR-04	**Violation Recording & Permit Verification**	Security staff shall be able to scan or manually enter a vehicle licence plate to verify the validity of its parking permit. The system shall allow security staff to log parking violations, including date, time, zone, and offence type.

2 Non-Functional Requirements

NFR-01	**Performance**	The system shall respond to permit verification queries within 2 seconds under normal load conditions (up to 500 concurrent users). Database queries for zone availability shall complete within 1 second.
NFR-02	**Security**	All user data and permit information shall be encrypted using AES-256 at rest and TLS 1.3 in transit. The system shall enforce role-based access control (RBAC) so that students, staff, and security personnel can only access their authorised functions.

TASK 2 — Workflow Diagram

The following diagram illustrates the end-to-end workflow of the Digital Parking Permit System, from initial vehicle registration through permit approval to final parking verification. It was designed using diagrams.net and exported as a PNG for inclusion in the repository under *diagrams/workflow_diagram.png*.



Workflow Summary: A user first registers their vehicle in the system. They then submit a permit request, which triggers a real-time zone availability check. If space is available, the permit is approved and the user can park. Security staff verify the permit on-site. If no space is available in the requested zone, the user is redirected to re-request with an alternative zone or date.

TASK 3 — Test Case Table

The following six test cases were designed in Google Sheets and exported here. They cover all required test types: two system test cases, two acceptance test cases, and two regression test cases.

Test ID	Feature	Input / Data	Expected Result	Test Type
TC-01	Vehicle Registration (System Test)	Valid licence plate: ABC-1234 User: student@univ.edu Vehicle: Toyota Corolla 2020	Vehicle is registered successfully. Confirmation email sent. Vehicle appears in user profile.	**System Test**
TC-02	Permit Approval Flow (System Test)	Registered vehicle submits permit request for Zone A, dates 01-Jun to 30-Jun	System validates eligibility, checks zone capacity, approves permit, and stores permit record in database.	**System Test**
TC-03	Student Permit Request (Acceptance Test)	Student selects Zone B, preferred dates, submits request via web portal	Student receives permit approval within 24 hours. Permit visible in 'My Permits' dashboard. Meets user expectation.	**Acceptance Test**
TC-04	Security Verification (Acceptance Test)	Security officer scans valid permit barcode for vehicle XYZ-5678 parked in Zone A	System displays 'PERMIT VALID' with expiry date. Security officer can log a violation if needed. Business requirement met.	**Acceptance Test**
TC-05	Vehicle Registration After Bug Fix (Regression Test)	Re-register a vehicle that was previously failing due to duplicate plate validation bug	System correctly identifies that the plate is already registered and shows appropriate error. No regression in registration logic.	**Regression Test**
TC-06	Permit Verification After Expiry Fix (Regression Test)	Valid permit with expiry date of today's date — verified by security at 09:00 AM	System correctly shows permit as VALID (not expired). Confirms fix for the 'valid permit shown as expired' defect remains in place.	**Regression Test**

■ System Test — checks the complete integrated system

■ Acceptance Test — checks user / business expectations

■ Regression Test — ensures prior fixes remain intact

TASK 4 — Defect Analysis

Defect Statement: "A valid parking permit is shown as expired during security verification."

Defect Category	**Logic / Data Validation Error**	The system incorrectly evaluates the permit expiry date, classifying a valid permit as expired. This is a logical defect in the date comparison or timezone handling rather than a cosmetic or configuration issue.
Severity	**High**	Security staff are unable to confirm the validity of genuine parking permits, which disrupts daily operations, causes delays for legitimate permit holders, and may result in wrongful enforcement actions.
Priority	**High — Fix Immediately**	Because this defect directly impacts a core operational function (permit verification) and affects real users on every verification attempt, it must be resolved before the next release or hotfix deployment.
Possible Root Cause	**Date/Time Comparison or Timezone Mismatch**	The most likely causes are: (1) The server stores expiry dates in UTC but the verification module compares them using local server time, causing off-by-one day errors; (2) The date comparison uses a less-than operator instead of less-than-or-equal, incorrectly treating today's expiry date as past; or (3) The expiry timestamp includes a time component (e.g., 00:00:00) that causes the permit to appear expired at the start of its final day.
Regression Test	**TC-06: Permit Verification After Expiry Fix**	After the fix is applied, re-run TC-06 to verify that a permit expiring on today's date is correctly shown as VALID when checked at 09:00 AM. Also re-run TC-01 through TC-04 to confirm no other verification functionality has been broken by the fix.

TASK 5 — GitHub Evidence

All project files, diagrams, test tables, and this PDF report have been uploaded to the GitHub repository listed below. The following checklist confirms completion of every required GitHub activity.

■ **Repository URL:** `https://github.com/[StudentName]/SER216_Final_Project_StudentName`

✓	Requirement	File / Reference	Notes
■	**Repository Created**	`SER216_Final_Project_StudentName`	Repository is public and correctly named.
■	**README.md Added**	`README.md`	Describes the project, tools used (Google Docs, Sheets, diagrams.net, GitHub), all file locations, and workflow summary.
■	**Feature Branch Created**	`feature/test-plan`	Branch created from main to isolate test plan work before merging.
■	**Minimum 3 Commits Made**	`Commit history (see screenshot)`	Commit 1: Add project report structure Commit 2: Add workflow diagram Commit 3: Add test case table and defect analysis Commit 4: Update README and upload final PDF
■	**Pull Request Created**	`PR: feature/test-plan → main`	Pull request opened with description of changes, reviewed before merge.
■	**Pull Request Merged**	`Merged to main branch`	All changes from feature/test-plan successfully merged into main.
■	**Project Files Uploaded**	`final_report.pdf, diagrams/, tables/, screenshots/`	All required files uploaded per the suggested repository structure.

Suggested Repository Structure

```
SER216_Final_Project_StudentName/
■■■ README.md
■■■ final_report.pdf
■■■ diagrams/
■ ■■■ workflow_diagram.png
■■■ tables/
■ ■■■ test_cases.xlsx
■■■ screenshots/
■ ■■■ repo_homepage.png
■ ■■■ commits.png
■ ■■■ branch.png
■ ■■■ pull_request.png
■■■ notes/
■■■ tools_used.md
```

Required GitHub Screenshots (Placeholders)

The following screenshot areas represent the evidence students must capture from their GitHub repository and insert into this report before submission.

#1	**Repository Homepage** Screenshot showing repository name, description, file list, and README preview.	*[Insert Screenshot Here]*
#2	**README.md File** Full README.md rendered view showing project description, tools, and file structure.	*[Insert Screenshot Here]*
#3	**Commit History (3+ commits)** Commits tab showing at least 3 meaningful commit messages with timestamps.	*[Insert Screenshot Here]*
#4	**Feature Branch** Branches tab or branch selector showing 'feature/test-plan' or similar branch.	*[Insert Screenshot Here]*
#5	**Pull Request** Open pull request from feature branch to main, with description.	*[Insert Screenshot Here]*
#6	**Merged Pull Request** Closed/merged pull request showing 'Merged' status badge.	*[Insert Screenshot Here]*
#7	**Final PDF in Repository** Repository file view showing final_report.pdf uploaded successfully.	*[Insert Screenshot Here]*